Solution: Median of Two Sorted Arrays

Problem Statement

Given two sorted arrays, `nums1` and `nums2` of different sizes in the ascending order, return the median of two sorted arrays after merging them.

The median is the middle value in an ordered set, or the average of the two middle values if the set has an even number of elements.

Examples

1. **Example 1**:
   - **Input**: `[1, 3, 5]` and `[2, 4, 6]`
   - **Expected Output**: `3.5`
   - **Justification**: When merged, the array becomes `[1, 2, 3, 4, 5, 6]`. The median is the average of the middle two values, `(3 + 4) / 2 = 3.5`.
2. **Example 2**:
   - **Input**: `[1, 1, 1]` and `[2, 2, 2]`
   - **Expected Output**: `1.5`
   - **Justification**: The merged array is `[1, 1, 1, 2, 2, 2]`. The median is `(1 + 2) / 2 = 1.5`.
3. **Example 3**:
   - **Input**: `[2, 3, 5, 8]` and `[1, 4, 6, 7, 9]`
   - **Expected Output**: `5`
   - **Justification**: The merged array would be `[10, 15, 20, 25, 30, 35, 40, 45]`. The median is the average of the middle two values, `25` and `30`, which is `(25 + 30) / 2 = 27.5`.

Solution

To find the median of two sorted arrays, we can use the merge step of the merge sort algorithm and the two-pinter approach. This approach involves merging the elements of both arrays into a single sorted sequence. However, unlike traditional merge sort, we don't need to merge the entire arrays. Instead, we only merge until we reach the median position.

The median position is determined by the combined size of the arrays, varying if the total number of elements is odd or even. The process involves comparing the elements at the pointer positions and advancing the pointer at the smaller element, thus mimicking a partial merge.

This step continues until the pointers reach the median position. The median is then calculated based on the last elements encountered by the pointers: directly for an odd total and as an average of the last two elements for an even total.

Step by Step Algorithm

1. **Initialize Pointers**: Start with two pointers, one for each array, both initialized to point to the first element of their respective arrays.
2. **Determine Median Position**: Calculate the position of the median in the merged array. If the total number of elements (N) in both arrays is odd, the median is at position `(N + 1) / 2`. If N is even, the median will be the average of the elements at positions `N / 2` and `(N / 2) + 1`.
3. **Simulate Merging**: Begin advancing the pointers through the arrays. At each step, compare the elements at the pointer positions in both arrays. Advance the pointer that points to the smaller element. This step mimics the merge process, but only the elements up to the median position are considered.
4. **Track Median Elements**: Keep track of the last two elements encountered by the pointers. These are necessary for calculating the median, especially in the case of an even total number of elements.
5. **Determine Median Value**: Once the pointers reach the median position(s):
   - For an odd total number of elements, the median is the last element moved.
   - For an even total number of elements, the median is the average of the last two elements moved.

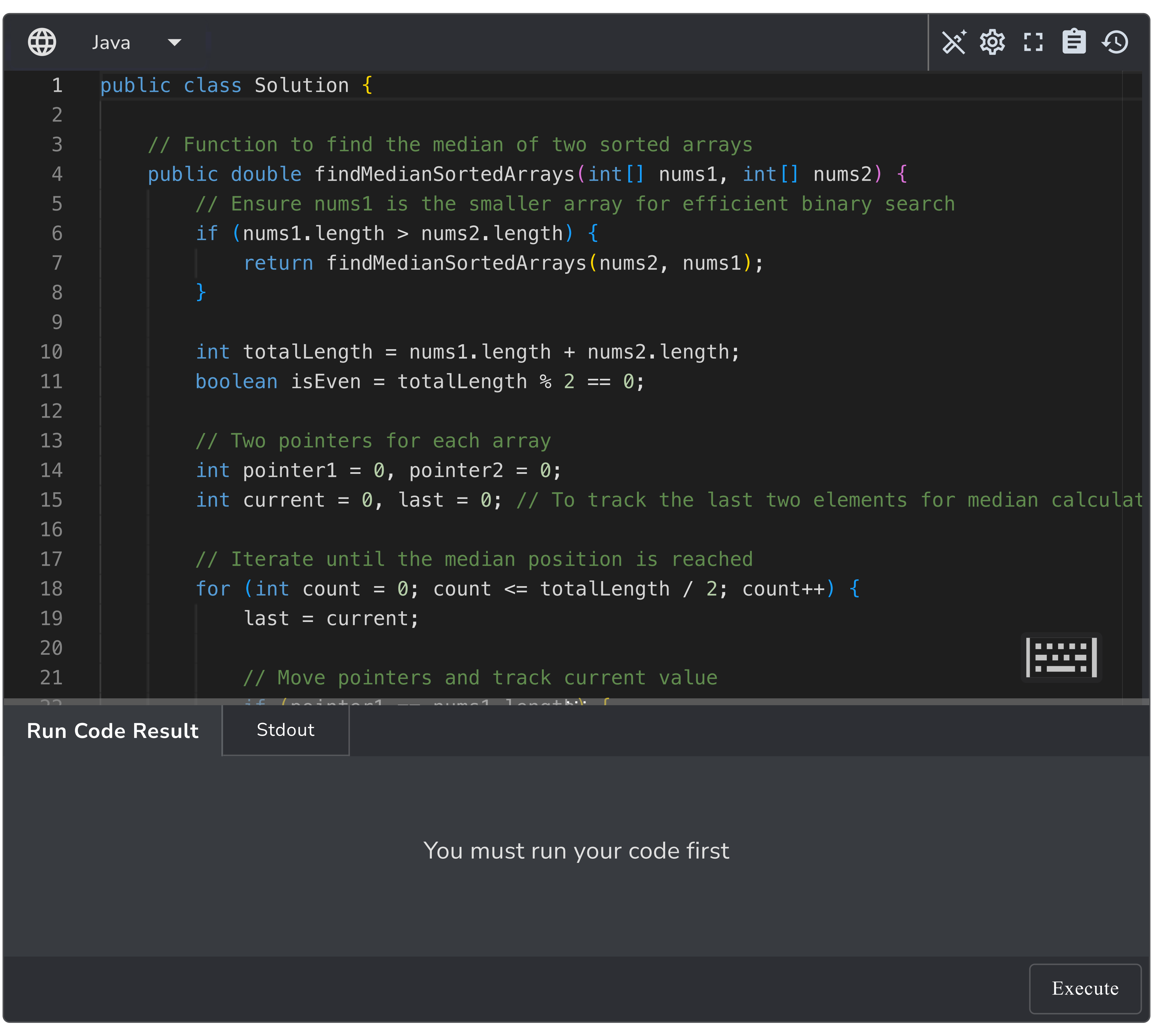
Algorithm Walkthrough

Let's illustrate the algorithm with an example. Consider two arrays `[2, 3, 5, 8]` and `[1, 4, 6, 7, 9]`.

1. **Initialize Pointers**: `pointer1 = 0` (for the first array), `pointer2 = 0` (for the second array).
2. **Median Position**: Total elements = 9, which is odd. Median position = `(9 + 1) / 2 = 5`.
3. **Simulate Merging**:
   - Compare `2` and `1`, move `pointer2`. New pointers: `pointer1 = 0`, `pointer2 = 1`.
   - Compare `2` and `4`, move `pointer1`. New pointers: `pointer1 = 1`, `pointer2 = 1`.
   - Compare `3` and `4`, move `pointer1`. New pointers: `pointer1 = 2`, `pointer2 = 1`.
   - Compare `5` and `4`, move `pointer2`. New pointers: `pointer1 = 2`, `pointer2 = 2`.
   - Compare `5` and `6`, move `pointer1`. New pointers: `pointer1 = 3`, `pointer2 = 2`.
4. **Track Median Elements**: The fifth element (median position) is `5`.
5. **Determine Median Value**: Since the total number is odd, the median is `5`.

Code

Here is the code for this algorithm:



```java
public class Solution {

    // Function to find the median of two sorted arrays
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        // Ensure nums1 is the smaller array for efficient binary search
        if (nums1.length > nums2.length) {
            return findMedianSortedArrays(nums2, nums1);
        }

        int totalLength = nums1.length + nums2.length;
        boolean isEven = totalLength % 2 == 0;

        // Two pointers for each array
        int pointer1 = 0, pointer2 = 0;
        int current = 0, last = 0; // To track the last two elements for median calculat

        // Iterate until the median position is reached
        for (int count = 0; count <= totalLength / 2; count++) {
            last = current;

            // Move pointers and track current value
```

You must run your code first

Time Complexity and Space Complexity Analysis

Time complexity

The time complexity of the provided code is **O(n + m)**, where (n) and (m) are the lengths of the two input arrays `nums1` and `nums2`. This is because the main loop in the function iterates approximately O((n + m)/2) times in the worst case, which simplifies to (O(n + m)) in Big O notation.

Space Complexity

The space complexity of the code is **O(1)** as the algorithm only uses a fixed number of extra variables (pointers and variables to track current and last values) regardless of the input size.

← Previous
Median Of Two Sorted Arrays
(Hard)

Next →
Longest Valid Parentheses
(Hard)

☐ Mark as Completed